Clinch: Sybil-proof token launches and batch swaps on Uniswap v4

Arnab Sengupta

ETHGlobal Mumbai, November 2026
Code: github.com/asengupta07/clinch

Working paper, written over two days. Version of 7 November 2026.

Abstract. Onchain identities cost nothing, so any token sale or batch auction that hands out supply by wallet count, by per-wallet caps or pro rata can be beaten by one person running many wallets. A recent result by Zhang, Li, Shi, Zhang, Xia and Han shows that adaptive clinching, the budget-constrained auction of Dobzinski, Lavi and Nisan, keeps truthful bidding as a dominant strategy even when a bidder splits its budget across any number of fake identities, as long as the aggregate budget it reports is enforceable. The same paper extends clinching to a two-sided market against a constant-product pool. We built the first implementation of both results. Clinch has two products: a token launch that sells a fixed supply on a rising price clock and then seeds a Uniswap v4 pool at the price it found, and a v4 hook that collects swaps into short block windows and clears each window with the two-sided mechanism plus one residual swap against the pool. Escrowing every budget up front is what makes the theorem's budget assumption hold on chain. This paper describes the mechanism as we implemented it, including the discretization and fixed-point choices the theory leaves open, and reports what we measured: no sybil split or misreport we could find gained more than 10^{-5} of the bidder's budget at a 0.1% price step, the Solidity engine matches a Python reference bit for bit, and on Sepolia a 1,000,000-token launch cleared in 99 ladder steps for 3.68M gas. We also explain where the guarantees stop, which matters as much.

1 Introduction

Most token launches are some kind of auction where bidders have budgets. They are run on chains where creating a new address is free. Put those two facts together and every rule that looks at addresses becomes something to game. A per-wallet cap invites fifty wallets. A pro-rata sale rewards whoever submits the most requests. Lotteries reward whoever buys the most tickets. Even a clean uniform-price auction, which does not care how many wallets you use, has a different problem: large bidders do better by reporting less demand than they have, the classic demand-reduction effect of multi-unit uniform-price auctions.

Swaps on automated market makers (AMMs) have a related problem in a different shape. A trade on a constant-product pool executes against whatever state the pool is in when the trade lands, so the order of trades inside a block is worth money [17]. Sandwich bots buy before a victim and sell after it. Batch auctions remove the ordering game by clearing many orders together [9, 15], but the usual batch rules (one uniform price, pro-rata fills) bring back the incentive to misreport.

What we wanted was a mechanism where three things hold at once without any identity layer: bidding your true value and budget is the best you can do, splitting into many wallets never helps, and nobody has to prove they are a person. The adaptive clinching auction of Dobzinski, Lavi and Nisan [2] was known to be truthful for budget-constrained bidders and Pareto optimal. Whether it survives sybil splitting was open until October 2026, when Zhang et al. [1] proved that it does, provided that a bidder cannot overreport its total budget. They also built an AMM version, building on the AMM mechanism-design framework of Chan, Wu and Shi [6], where the pool's pricing curve plays the role of supply, and showed that combined with a two-sided wrapper it is sybil-robust and weakly Pareto optimal, which they argue is about the best any AMM mechanism can do.

The budget condition is the part that blockchains handle well. If a bid only counts after its budget is locked in a contract, a reported budget is a deposited budget, and nobody can overreport. So the theorem's main assumption becomes a line of Solidity.

What we built. Clinch is a working implementation of [1] on Uniswap v4:

- **ClinchLaunch**, a token sale. Bidders escrow a budget and a maximum price, the price rises on a ladder, and each bidder clinches the supply that the others can no longer afford. When the sale settles, part of the proceeds and the unsold tokens seed a full-range v4 pool at the clearing price, with the liquidity locked.
- **ClinchHook**, a v4 hook. Direct swaps are disabled. Orders are escrowed into windows of a few blocks, and each window is cleared by the two-sided clinching mechanism with one residual swap against the pool.
- A shared Solidity library, **ClinchEngine**, that both contracts use, checked against a Python reference engine and a TypeScript port in the frontend.

What this paper adds to [1]. The paper describes a continuous price clock over real numbers. A contract needs a discrete ladder, integer arithmetic, a rule for which way to round, a bound on how long the loop runs, and a way to settle against a real v4 pool that rounds and may charge a protocol fee. We describe those choices in Section 4, and Section 7 reports how much they cost in incentives and in gas. Some of what we found changed how we pitch the project. The most important one: splitting a budget does not give the same result as bidding once. It usually gives strictly less (Section 7.3), which is consistent with the theorem but not with the slogan we started with.

2 Background

2.1 Adaptive clinching

Ausubel's clinching auction [3] sells multiple units with an ascending price. At each price, a bidder "clinches" any units that the other bidders' total demand can no longer cover, and pays the current price for them. Dobzinski, Lavi and Nisan [2] adapted this to bidders with hard budget limits and a divisible good. Demand at price p is the remaining budget divided by p, so demand falls as the price rises and as budgets get spent. They showed that this adaptive clinching auction is truthful and Pareto optimal when budgets are public, and Bhattacharya, Conitzer, Munagala and Xia [4] extended truthfulness to private budgets that can only be under-reported.

None of these results says anything about a bidder who submits several identities. Sybil-proofness (also called false-name-proofness) has a long history [11, 12], and many otherwise good mechanisms fail it. That is the gap [1] closes.

2.2 The result we implement

Zhang et al. [1] introduce a target-based framework for clinching. Instead of a fixed supply, the auction has a target that can depend on the auction's own state, and they prove the incentive results at that level of generality. It also covers clinching with supply that arrives over time, as in Goel, Mirrokni and Paes Leme [8]. The two cases that matter here are:

- **Fixed supply** (their Theorem 1.3). In the adaptive clinching auction for a fixed divisible supply, bidding truthfully as a single identity weakly dominates every strategy that replaces the bidder with finitely many identities, with arbitrary reported values and aggregate declared budget at most the true budget.
- **AMM supply** (their Theorems 1.1, 4.19 and 6.6). With the pool's cost curve as supply and a two-sided wrapper, the mechanism is individually rational, settles exactly against the pool, is sybil-robust incentive compatible, and is weakly Pareto optimal for eligible bids. The opposite-

direction budgets of sybil identities may be unbounded; only the same-direction aggregate must be enforceable.

They also prove (Theorem 1.2, Section 7) that if budgets can be overreported, no deterministic AMM mechanism gets sybil-robust incentive compatibility and weak Pareto optimality together, even without any sybil identities. Escrow is therefore not a convenience. It is required.

2.3 Uniswap v4 and existing launch designs

Uniswap v4 [13] keeps all pools in one PoolManager contract and lets each pool attach a hook contract that is called at fixed points (before and after initialize, add or remove liquidity, swap). All token movements happen inside an `unlock` callback, and balances are netted at the end. A hook that rejects every swap that it did not start itself can therefore route all trading through its own logic, and still use the pool for the final settlement swap.

Uniswap's own launch product, the Continuous Clearing Auction (CCA) [10], runs a uniform-price auction over time and seeds a pool when it ends. It deals well with timing games, but a uniform-price auction is not truthful: as noted above, big bidders gain by shading. Clinch is a direct alternative with a different trade-off, which we compare in Section 7.3.

3 Model

There is one sale asset (the token being sold, or token0 of a pool) and one payment asset (USDC in our deployment, or token1). Prices are always "payment asset per sale asset".

A bidder i has a private value V_i per unit and a hard budget Q_i in the payment asset. It reports a limit v_i (the highest price it will pay) and a budget q_i. If it receives x_i units and pays y_i, its utility is

$$U_i = V_i x_i - y_i \tag{1}$$

when $y_i \le Q_i$, and minus infinity otherwise. For two-sided batches a trader on the other side (a seller) is described the same way with the assets swapped: a minimum price and an amount of the sale asset.

A **strategy** for a bidder is any finite set of identities, each with its own reported limit and budget. The bidder's utility is computed from the aggregate flows of all its identities. Following [1], we allow any reports subject to one constraint: the same-direction budgets of a bidder's identities must sum to at most Q_i. In the two-sided setting, identities placed on the opposite side have unbounded budgets.

Sybil-robust incentive compatibility means that for every bidder and every profile of the other bids, no strategy gives higher utility than a single identity reporting (V_i, Q_i):

$$U_i(\text{any identity set}) \le U_i(\text{truthful single bid}). \tag{2}$$

Escrow. In Clinch a bid or order only exists after its full budget has been transferred into the contract. A bidder therefore cannot report more budget than it controls, and the aggregate constraint holds by construction. The one assumption we cannot enforce in code is censorship resilience: a block builder who can drop some bids and not others is outside the model, as it is in [1] and in Otter [5].

4 The mechanism

This section follows Sections 3 to 5 of [1] and adds the choices an implementation has to make. All quantities below are "remaining" quantities that change as the clock runs.

4.1 Demand and clinching

Bid i has limit v_i and remaining budget b_i (initially q_i). At clock price p its demand is

$$D_i(p) = \begin{cases} b_i/p & p < v_i \\ 0 & p > v_i \end{cases} \tag{3}$$

and at $p = v_i$ the bid is exiting, which the stopping rule handles separately. Given residual supply $S(p)$, bid i clinches what the others can no longer absorb:

$$c_i(p) = \max\left(0, S(p) - \sum_{j \neq i} D_j(p)\right). \tag{4}$$

We use the equivalent water-level form from [1], which is easier to compute and makes order independence obvious. Let $B = \sum_j b_j$ be the total remaining budget of active bids and define the water level

$$h = B - pS. \tag{5}$$

Expanding Equation 4 with $D_j = b_j/p$ gives $c_i > 0$ exactly when $b_i > B - pS = h$, and the amount is

$$\text{buy}_i = \frac{b_i - h}{p}. \tag{6}$$

After these purchases every active budget satisfies $b_i \leq h$, and the bids with $b_i = h$ are the current clinchers. A purchase of δ units at price p lowers B by $p\delta$ and lowers pS by $p\delta$ as well, so h does not move. This is why the order in which purchases at one price are made has no effect on the outcome, and why the result does not depend on bid order (Remark 2.5 of [1]).

4.2 Supply: fixed, or the pool's chord target

Launches sell a fixed supply S_0. After allocating X units in total, $S = S_0 - X$.

Pools have no fixed supply. For a constant-product pool with reserves (x_0, y_0) the cost of buying z units of the sale asset, and the marginal price at that point, are

$$C(z) = \frac{y_0 z}{x_0 - z}, \qquad C'(z) = \frac{x_0 y_0}{(x_0 - z)^2}, \qquad p_0 = \frac{y_0}{x_0}. \tag{7}$$

Let X and Y be the units allocated and the payment collected so far. At price p the supply target is the furthest point where a line of slope p through (X, Y) still lies on or above the cost curve (equation (11) of [1]):

$$T(p, X, Y) = \max\{z \in [X, x_0) : C(z) \leq Y + p(z - X)\}, \qquad S = T - X. \tag{8}$$

The idea is that the mechanism can promise at price p any amount the pool could deliver if every further unit were paid for at p. Writing $K = Y - pX$ and multiplying out Equation 7 (with $x_0 - z > 0$), the condition $C(z) \leq K + pz$ becomes

$$pz^2 + (y_0 + K - px_0)z - Kx_0 \leq 0, \tag{9}$$

so T is the larger root of that quadratic. With $b = y_0 + K - px_0$ and discriminant $D = b^2 + 4pKx_0$ we evaluate the root in the form that avoids cancellation:

$$T = \begin{cases} \dfrac{2Kx_0}{b + \sqrt{D}} & b \geq 0 \\ \dfrac{\sqrt{D} - b}{2p} & b < 0 \end{cases}. \tag{10}$$

A purchase of δ at price p moves the state to $(X + \delta, Y + p\delta)$, which leaves $K = Y - pX$ and therefore T unchanged, and lowers S by exactly δ. So clinching against the pool behaves exactly like clinching against a fixed supply between price steps, and at termination the total payment sits on the curve: $Y_f = C(X_f)$.

Our first draft used a simpler target, $X - Y/p$ evaluated on updated reserves. It looks reasonable and is wrong: in simulation it withheld up to 20% of the supply the correct target would sell (median

2.4%), and its incentive error was roughly 5ε instead of well below ε. We mention it because it is an easy mistake to make.

4.3 Two-sided batches

In a pool batch there are buyers (they pay token1, and their limit is a maximum price) and sellers (they pay token0, and their limit is a minimum price). Only orders that are marketable at the current spot price p_0 count: buyers with $v_i \geq p_0$ and sellers with $u_i \leq p_0$. The contract enforces this when an order is placed. Measured in token0 at spot, the two sides want

$$D_{\text{buy}} = \sum_{i \in E_{\text{buy}}} \frac{q_i}{p_0}, \qquad D_{\text{sell}} = \sum_{i \in E_{\text{sell}}} r_i, \qquad M = \min\big(D_{\text{buy}}, D_{\text{sell}}\big). \tag{11}$$

The smaller side is the minority (sellers on a tie). It is filled completely at spot: a minority seller receives $r_i p_0$ and a minority buyer receives q_i/p_0. The minority's volume M is then folded into the cost curve the dominant side bids against (equation (57) of [1]):

$$\tilde{C}_M(z) = \begin{cases} p_0 z & 0 \leq z \leq M \\ p_0 M + C(z - M) & M \leq z < M + x_0 \end{cases}. \tag{12}$$

The dominant side runs the one-sided auction of the previous two subsections against $\tilde{C}_M$. For constant product the extended target is M plus the root of Equation 9 with $K = Y - pX + (p - p_0)M$. When sellers dominate, the roles of the assets swap: the clock runs in token0 per token1 starting at $1/p_0$, and a seller's minimum price u becomes a maximum of $1/u$ in reversed units. Only the residual touches the pool:

$$\text{pool releases } X_f - M, \qquad \text{pool receives } Y_f - p_0 M. \tag{13}$$

Matched volume never moves the pool price, which is why a sandwich bot has nothing to sandwich (Section 7.4).

4.4 The discrete ladder

The continuous clock becomes a geometric ladder with step ε. To keep the proportional exit rule exact, we cap each step so that every bidder's limit lands exactly on a step:

$$p_{k+1} = \min(p_k(1 + \varepsilon), \min\{v_i : v_i > p_k, b_i > 0\}). \tag{14}$$

At each step the engine recomputes T and S, splits the remaining bids into active ($v_i > p$) and exiting ($v_i = p$), and checks whether active demand now fits. If it does, the auction stops. Active bids receive their full demand and the leftover L is shared among exiting bids in proportion to budget:

$$\sum_{\text{active}} \frac{b_i}{p} \leq S \Longrightarrow x_i \leftarrow x_i + \frac{b_i}{p}, \quad x_j \leftarrow x_j + \frac{b_j}{B_E} \cdot \min\left(L, \frac{B_E}{p}\right), \quad L = S - \sum_{\text{active}} \frac{b_i}{p}, \tag{15}$$

where B_E is the total budget of exiting bids. If it does not fit, exiting budgets are refunded and the restoration purchases of Equation 6 are made at the current price, and the clock moves on.

4.5 How long the loop runs

A contract cannot run an unbounded loop, so we need a bound on the number of steps. Lemma 6.2 of [1] shows that with total budget B_0 the clock never passes $P_B = C'(C^{-1}(B_0))$. For constant product, $C^{-1}(B) = Bx_0/(y_0 + B)$, so $x_0 - z = x_0 y_0/(y_0 + B)$, and

$$P_B = \frac{(y_0 + B_0)^2}{x_0 y_0}. \tag{16}$$

A geometric ladder from $p_0 = y_0/x_0$ reaches P_B after $\ln(P_B/p_0)/\ln(1 + \varepsilon)$ steps, and each limit can add one extra capped step, so

$$\text{steps}_{\text{hook}} \le \frac{2\ln(1+B_0/y_0)}{\ln(1+\varepsilon)} + n + 1, \qquad \text{steps}_{\text{launch}} \le \frac{\ln(p_{\max}/p_{\text{res}})}{\ln(1+\varepsilon)} + n. \tag{17}$$

In 700 simulated runs the final price never exceeded $P_B(1+\varepsilon)$ and the step count never exceeded the hook bound. A fixed-supply launch with unbounded limits is a different story: allocations only converge as $p \to \infty$, and one test run climbed to about 10^7 times the reserve over 185,000 steps. So every launch has a price ceiling $p_{\max}$, limits above it are clamped, and at $p_{\max}$ the stopping allocation runs. The factory also rejects any launch whose worst case, (ladder steps plus one exit step per bid) times the bid cap, is too large to settle in one block.

4.6 Settlement and the surplus rebate

On the ladder, buyers pay the top of each step. The engine's payments therefore leave the pool about 0.9ε of volume above its curve, which is a real cost to traders. The hook does not replay the engine's exact allocation against the pool. It swaps everything the dominant side paid (minus minority payouts) into the pool as an exact-input swap, then splits what the pool actually returned in proportion to engine allocations:

$$\text{poolIn} = Y_f - \sum \text{minority payouts}, \qquad \text{payout}_i = x_i \cdot \frac{M + \text{poolOut}}{X_f}. \tag{18}$$

Because the pool returns at least what the engine allocated, every dominant-side order gets a small rebate (we observed +0.0028% on a live batch). This departs from the exact settlement of [1], so we treat it as our own design choice and checked it separately: in our adversarial search the worst deviation gain went down with the rebate, from 2.8×10^{-3} to 5.3×10^{-4} of the budget at $\varepsilon = 1\%$. The settlement check tolerates v4′s rounding and any protocol fee f (in pips):

$$\text{poolOut} \cdot 10^6 + 16 \cdot 10^6 \ge (X_f - M)(999\,999 - f). \tag{19}$$

4.7 Fixed point and rounding

On chain every price is a WAD (scaled by 10^{18}) ratio of raw token units. Quantities round down and payments round up. Inside Equation 10 every rounding has to push z down, otherwise the pool can be underpaid by a wei or two: K rounds down, b rounds up, the negative term of the discriminant rounds toward $-\infty$, the first branch uses $\lceil\sqrt{D}\rceil$ and the second $\lfloor\sqrt{D}\rfloor$, and a negative discriminant (which happens at tangency) is clamped to zero. Our first integer version got this wrong and underpaid the pool by 1 to 2 wei in 26 of 300 runs. After the fix there were none in 2,000 runs, across 18 orders of magnitude of reserves and 12 of price, and the largest intermediate value needed 235 bits.

When a launch seeds its pool, it converts the clearing price to v4′s square-root format, $\sqrt{P} \cdot 2^{96}$, computed as $\sqrt{p \cdot 2^{192}}$ with a 512-bit intermediate. The hook reads full-range reserves back from the pool state as $r_0 = L \cdot 2^{96}/\sqrt{P}$ and $r_1 = L \cdot \sqrt{P}/2^{96}$.

5 What is proven and what is measured

Here is which guarantees come from [1] and which ones we only measured.

Property	Source	Status in Clinch
Truthful bidding dominant, sybil splits never help (launch)	Theorem 1.3 of [1]	Proven for the continuous clock; ε-approximate on our ladder, measured
Same, for the two-sided pool mechanism	Theorems 4.19, 6.6 of [1]	Same as above
Weak Pareto optimality for eligible bids	Theorems 4.13, 6.7 of [1]	Inherited from the mechanism; not separately tested
Exact settlement, individual rationality	Theorems 4.11, 6.3 of [1]	Tested by fuzzing; the rebate makes settlement exact against the real pool
No risk-free profit from a batch	Fact 2.9 of [1]	Tested; zero cases found
Termination within P_B	Lemma 6.2 of [1]	Measured; 0 violations in 700 runs
Allocation independent of bid order	Remark 2.5 of [1]	Tested by fuzzing
Budgets cannot be overreported	Assumption of [1]	Enforced by escrow
No censorship of bids	Assumption of [1]	Not enforceable by the contracts

Table 1: Guarantees and where they come from.

The discrete ladder means incentive compatibility only holds up to about ε of the bidder's budget. We do not have a proof of that bound. The data in Section 7 suggests the real error is much smaller, roughly second order in ε, but it remains an empirical claim and we present it that way.

Two things the theorem does **not** say, and which we had wrong at first:

1. **Splitting does not give the same outcome.** Truthful bidding weakly dominates; it does not mean every split is equal. In our whale experiment a bidder's utility fell from 680 with one wallet to 270 with 100 wallets. So the accurate slogan is "splitting never helps, and usually hurts".
2. **Clinch does not limit whales.** The guarantee is about incentives, not about distribution. In the same instance the truthful whale won 87% of the supply under Clinch against 71% under a uniform-price auction. If someone wants per-person limits on top, they need identity, which is what Section 8 adds.

6 Implementation

6.1 Contracts

All allocation logic lives in one library, `ClinchEngine`, with three entry points: `target()` (equations Equation 10 and its extended form), `run()` (the one-sided ladder) and `twoSided()` (the minority fill and the dominant-side auction). `ClinchLaunch` and `ClinchHook` wrap it and only touch Uniswap inside `settle()`.

ClinchHook uses three hook permissions, `beforeInitialize`, `beforeAddLiquidity` and `beforeSwap`, which give the address flags `0x2880`; the address is mined with CREATE2. Pools can only be initialized by the hook itself, with fee 0, and liquidity must be full range so that the pool is exactly $xy = k$ and Equation 7 is the true cost curve. `beforeSwap` reverts any swap whose sender is not the hook (`DirectSwapDisabled`). Orders go into the current batch; the first order after a window closes opens the next one. An order must be marketable at spot, at least the pool minimum, and keep its side of the batch under a depth cap (5% of reserves in our deployment). `settle()` is permissionless: it runs the engine on the oldest closed batch, makes the single residual swap inside `unlock`, and stores every allocation. Anyone can then `claim` on behalf of any owner. If a batch cannot be settled within a rescue window (1,800 blocks), anyone can `cancelStuck` and every order is refunded in full. A batch is also cancelled with full refunds if liquidity is removed so far that the batch exceeds twice the depth cap.

ClinchLaunch is created by `ClinchLaunchFactory`, which validates the parameters (reserve below ceiling, ε between 0.01% and 10%, bid cap at most 200, the work bound above), reserves the launch's v4 pool key on the hook in the same transaction so nobody can initialize it first, and moves the sale tokens in. Bidding is open until the end time. `settle()` runs the ladder, then seeds a full-range pool at the clearing price with a set share of the proceeds plus the reserved and unsold tokens; the LP position

stays in the launch contract. If nobody settles within the rescue delay, anyone can cancel and every bid is refunded.

Neither contract has an owner or an upgrade path. The factory's only privileged call is a one-time `setHook`.

6.2 Off-chain parts

There are two more engines besides the Solidity one. A Python reference engine (float and integer versions) was used for all the research in Section 7, and Foundry tests call it through FFI to check that the Solidity engine produces identical integers. A TypeScript port runs in the browser. The app uses it to estimate "if this sold now" before a bid is placed, and to replay a settled launch step by step and confirm the replay matches what the contract stored. A small keeper script watches for closed batches and ended launches and settles them, but nothing depends on it: settlement can be called by any wallet or straight from a block explorer. Chain reads can go through the Ethereum Foundation's Anon-RPC [14] so that the RPC provider does not learn which markets a user is looking at; transactions still go to a normal RPC.

7 Evaluation

All numbers below come from the repository's research scripts and test suite, or from transactions on Sepolia. The simulations use random instances, not historical data.

7.1 Incentives under the discrete ladder

We searched for profitable deviations with random search plus hill climbing: 200 random instances, 400 adversarial evaluations each, per mode. The table reports the largest gain found, as a fraction of the deviating bidder's budget.

Deviation	$\varepsilon = 1\%$	$\varepsilon = 0.1\%$	**Cases** $> \varepsilon$
Pool, sybil split into up to 8 wallets	2.8×10^{-3}	1.5×10^{-5}	0
Pool with the surplus rebate	5.3×10^{-4}	3.6×10^{-6}	0
Pool, value misreport	2.5×10^{-3}	9.5×10^{-6}	0
Launch, sybil split	2.8×10^{-5}	8.5×10^{-8}	0
Two-sided, sybils on the opposite side (unbounded budgets)	1.7×10^{-3}	4.2×10^{-6}	0
Two-sided, free-lunch bot	0	0	0
Control: our first-draft target	5.0×10^{-2}	5.0×10^{-3}	2

Table 2: Largest deviation gain found, as a fraction of budget.

Every mode stays under ε, and the gain falls faster than ε does: a tenfold smaller step gives a 100 to 300 times smaller gain. The control row shows that the search does find violations when the target is wrong. This is a search, not a proof, so structured attacks (for example identities placed exactly at other bidders' limits) are covered only by chance.

7.2 Invariants and differential tests

Over 2,000 random instances at $\varepsilon = 0.1\%$, over-allocation, budget violations, dependence on bid order, prices below spot and two-sided settlement error were all at float noise ($\leq 2 \times 10^{-13}$), and individual rationality violations were exactly zero. The integer engine had zero pool underpayments, zero budget or rationality violations and zero overflows in 2,000 runs. The Solidity engine matched the Python integer engine bit for bit (same allocations, payments, final price and step count) over 768 fuzzed differential runs.

The contract suite has 99 tests, including fuzzing at 2,000 runs per property in CI. They cover the full lifecycle of launches and batches, every revert path in the hook, conservation of tokens in the hook

(dust at most 10^9 wei), and regression tests for eight liveness problems found in an internal review: batches that could not fit in a block, liquidity removal mid-batch, the v4 protocol fee, very small pools, low-precision spot prices, slot griefing with ineligible orders, unvalidated launch pool configurations, and pool-key griefing.

7.3 Comparison with other launch designs

We compared designs on one instance: a whale with a budget of 1,500 and a value of 2.50 against 30 honest bidders, 1,000 tokens, reserve 1.00.

Design	**Honest bidding best?**	**Does splitting help?**	**No ID needed?**
Uniform price, no cap (CCA style)	No: shading to 67.5% of budget and 80% of value raises utility from 276 to 396 (+44%)	No: utility flat at 276	Yes
Fixed price with per-wallet cap	No	Yes: 40 with one wallet, 602 with 50	Yes
Uniform price with per-wallet cap	No	Yes: 49 with one wallet, 361 with 50	Yes
Caps plus proof of personhood	No: shading still pays under the cap	No	No
Clinch	Yes: truthful report (680) was the best found	No, it hurts: 680 to 270 at 100 wallets	Yes

Table 3: Whale experiment. Utility in units of the payment asset.

Two things here surprised us. Splitting does not help in an uncapped uniform-price auction at all; it is the per-wallet cap that makes splitting pay. And the problem with uniform price is truthfulness, not sybils. Clinch is the only row that is both truthful and needs no cap.

7.4 Sandwich attacks

Against a plain constant-product pool with reserves 10,000 / 10,000, a bot that front-runs and back-runs a victim earned:

Victim trade	**Slippage**	**Plain pool, bot profit**	**Clinch, bot profit**
100	0.5%	0.51	0
500	1%	5.25	0
500	3%	15.7	0
1,000	5%	54.7	0

Table 4: Sandwich profit in units of the payment asset.

In a Clinch batch all orders see the same clock, matched volume does not move the pool, and order within the window does not matter, so there is nothing to sandwich.

7.5 Gas

Settlement costs roughly 1,200 to 1,800 gas per bid per ladder step.

Case	Bids	ε	Steps	Settle gas
Hook, batch about 5% of pool	32	0.1%	n/a	1.38M
Hook prototype, batch 1% of pool	50	0.1%	10	0.84M
Hook prototype, batch 10% of pool	50	0.1%	110	10.2M
Launch, price range 10x	40	1%	205	10.0M
Launch prototype, contested, range 3x	50	1%	106	5.8M
Launch prototype, contested, range 3x	50	0.1%	874	47.5M
Sepolia: launch, 6 bids	6	1.5%	99	3.68M
Sepolia: two-sided batch, 3 orders	3	0.1%	2	1.02M

Table 5: Settlement gas. The 47.5M case is rejected by the factory's work bound.

The practical defaults follow from this: a 0.1% step on the hook (with the rebate), and a 1% to 1.5% step plus a price ceiling for launches. Very large contested launches would need the ladder split across several transactions, which we have not built.

On Sepolia, writing a new storage slot appears to cost about 100,000 gas instead of the usual 20,000 to 22,100: a fresh token allowance cost 128k gas, and placing a batch order cost 456k to 859k. We have not confirmed this against Sepolia's fork specification. If mainnet ever prices state growth the same way, the order record (owner, flags, limit, budget, spent, received) should be packed into fewer slots.

7.6 On Sepolia

We deployed everything against Uniswap's official Sepolia PoolManager (`0xE03A…3543`) and verified all contracts on Etherscan.

- A launch of 1,000,000 tokens with six bidders settled at a clearing price of $0.42, raising 252,631.85 USDC in 99 ladder steps for 3.68M gas. The in-browser replay matches the stored allocations exactly. It then seeded a CLNCH/USDC pool at $0.42.
- A five-order two-sided WETH/USDC batch settled in one transaction for 1.32M gas, moving spot from $3,000 to $3,048.92.
- A later three-order WETH/USDC batch (one seller of 1 WETH, two buyers with 2,300 USDC) cleared with buyers averaging $3,048.80 and the seller $3,048.73 against a spot of $3,048.92. The 0.2456 WETH left over went to the pool.
- The first batch on the launch-seeded CLNCH/USDC pool cleared at exactly $0.4200 for the buyers. Both batches were settled and claimed by a wallet with no orders in either.
- A direct swap through Uniswap's own test router reverted with `DirectSwapDisabled`, and a concentrated-range liquidity add reverted with `FullRangeOnly`. In all, nine failure paths gave the expected error.

8 A per-person cap on top: the World ID arm

Clinch does not need identity, but some issuers will still want per-person fairness, for example a cap on how much any one human can buy. To show that the two compose, we built a comparison arm that runs the same launch with a per-human cap using World ID [16], without changing any core contract.

The launch is created with a contract called `HumanGate` as its payment asset. HumanGate is a 1:1 voucher for USDC (hUSDC) that only a verified human can mint, up to 2,000 USDC in total per human. A human proves once with a World ID Proof of Human credential whose signal is their wallet address, which binds the nullifier (World ID's per-person, per-action identifier) to that wallet. A second wallet presenting the same nullifier is rejected. hUSDC can only move into the sale, the pool or the hook and back, so a verified person cannot pass their allowance to other wallets. Bidding, settlement and claims are unchanged, and hUSDC redeems 1:1 for USDC at any time.

World ID 4.0 proofs are verified by World's Developer Portal; there is no 4.0 verifier contract on Ethereum Sepolia. So our server forwards each proof unchanged to the portal, checks that its signal

matches the requesting wallet, and signs an attestation of (chain, verifier, signal hash, nullifier, external nullifier). A contract, `AttestedWorldID`, checks that signature on chain. The server can only vouch for proofs the portal accepted, and the one-wallet-per-nullifier rule is enforced by the contract, so the server cannot give one person two wallets. On a chain with a World ID verifier contract the same gate takes proofs directly.

We tested this on Sepolia with the World ID simulator: a real staging proof passed the portal and registered wallet A, which then deposited and bid. The same simulated person on a second wallet was refused, and replaying A's attestation from another wallet reverted with `InvalidProof`. The trade-off is plain: a cap needs a credential and excludes people without one. Under the cap, Clinch's incentive properties still hold, which is not true of the capped uniform-price sales above.

9 Limitations and open questions

- **Incentives are ε-approximate.** The bound is measured, not proven.
- **Censorship.** A builder who can drop chosen bids breaks sybil-robustness. This is assumed away in [1] and we inherit the assumption.
- **Weak, not full, Pareto optimality.** Some reverse-direction trades with the pool are left on the table. [1], [5] and [7] argue this is about the best achievable together with sybil-robustness.
- **Prices are not uniform.** Each unit is paid at the price where it was clinched, so two bidders can pay different average prices. This is inherent to clinching.
- **Capital is locked** for the whole window. Batches are short; launches last days.
- **Bids within ε of spot** can see an allocation error of order one, because the first step jumps straight to their limit. Feasibility and incentives still hold. A finer ladder near spot would fix it and is not implemented.
- **Large launches** can exceed one block of gas. Splitting the ladder across transactions, or an optimistic solver with a challenge window, would solve this. A verify-only shortcut does not: the terminal certificate (Lemma 4.12 of [1]) certifies efficiency, and many efficient outcomes are not the clinching outcome, so accepting any certified outcome would lose sybil-proofness.
- **Griefing.** One actor can fill the order or bid slots with minimum-size entries. Minimum sizes raise the cost but do not remove it.
- **Liquidity providers earn no swap fees**, since hook pools are fee 0 so that the pool is exactly $xy = k$.
- **Not audited.** This is hackathon code with an internal review, regression tests and fuzzing.

10 Conclusion

The result of Zhang et al. [1] turns a long-standing question in auction theory into something you can deploy: adaptive clinching stays truthful under sybil splitting, and escrow is enough to meet its one hard assumption. Clinch shows that it fits on Uniswap v4 at a reasonable cost, for both a token launch and a batch market, and that the gaps between the continuous theory and an integer contract (ladder, rounding, termination, settlement against a real pool) can be closed without visibly weakening the incentives. The parts we would work on next are a proof of the ε bound, a multi-transaction ladder for large launches, and a closer look at storage costs.

Acknowledgements. Thanks to the authors of [1] for a paper detailed enough to implement from, and to the Uniswap, Ethereum Foundation and World teams whose tooling we used.

References

[1] C. Zhang, Y. Li, E. Shi, M. Zhang, L. Xia, Q. Han. Sybil-Proofness of Generalized Adaptive Clinching and Implications for Blockchain Mechanism Design. arXiv:2610.36629, October 2026. Also Cryptology ePrint Archive 2026/2674.

[2] S. Dobzinski, R. Lavi, N. Nisan. Multi-unit auctions with budget limits. In *Proceedings of FOCS*, 2008. Journal version in *Games and Economic Behavior* 74(2), 2012.

[3] L. M. Ausubel. An efficient ascending-bid auction for multiple objects. *American Economic Review* 94(5), 2004.

[4] S. Bhattacharya, V. Conitzer, K. Munagala, L. Xia. Incentive compatible budget elicitation in multi-unit auctions. In *Proceedings of SODA*, 2010.

[5] E. Shi, M. Zhang, H. Chung, Y. Li. Otter: A provably MEV-resilient AMM via surplus redistribution. arXiv:2609.03474, September 2026.

[6] T. Chan, K. Wu, E. Shi. Mechanism design for automated market makers. In *Proceedings of AFT*, 2025.

[7] Y. Li, E. Shi, M. Zhang. A trilemma in AMM mechanism design. In *Financial Cryptography (FC)*, 2026.

[8] G. Goel, V. Mirrokni, R. Paes Leme. Clinching auctions with online supply. In *Proceedings of SODA*, 2013.

[9] A. Canidio, R. Fritsch. Arbitrageurs' profits, LVR, and sandwich attacks: batch trading as an AMM design response. 2024.

[10] Uniswap Labs. Continuous Clearing Auction. Documentation and whitepaper, November 2025. https://docs.uniswap.org/contracts/liquidity-launchpad/CCA

[11] M. Yokoo, Y. Sakurai, S. Matsubara. The effect of false-name bids in combinatorial auctions: new fraud in internet auctions. *Games and Economic Behavior* 46(1), 2004.

[12] B. Mazorra, N. Della Penna. The cost of sybils, credible commitments, and false-name proof mechanisms. arXiv:2301.12813, 2023.

[13] H. Adams, M. Salem, N. Zinsmeister, S. Reynolds, A. Adams, W. Pote, M. Toda, A. Henshaw, E. Williams, D. Robinson. Uniswap v4 Core. Whitepaper, 2024. Developer documentation at https://docs.uniswap.org

[14] Ethereum Foundation. Anon-RPC. https://ethereum.github.io/anon-rpc/

[15] E. Budish, P. Cramton, J. Shim. The high-frequency trading arms race: frequent batch auctions as a market design response. *Quarterly Journal of Economics* 130(4), 2015.

[16] World. World ID documentation: IDKit integration and the Proof of Human credential. https://docs.world.org/world-id

[17] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, A. Juels. Flash Boys 2.0: frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *IEEE Symposium on Security and Privacy*, 2020.

Appendix A. Deployment and parameters

All addresses are on Ethereum Sepolia (chain 11155111) and verified on Etherscan.

Contract	**Address**
PoolManager (Uniswap)	`0xE03A1074c86CFeDd5C142C4F04F1a1536e203543`
ClinchHook	`0x3e755Ab2f2784b7B6e66B2Ac302a1Ab2a0a26880`
ClinchLaunchFactory	`0x065AeDe864D1Fd2a8c0044dD859e50eff1927694`
ClinchLaunch (settled)	`0x399C300b8D7E18B5DEc2D7dA503c3EE0efE6e97c`
HumanGate (World ID arm)	`0x938c55747b018dA7fbE2BB59ad24f58b7AED2247`
AttestedWorldID	`0x28C2FBBFb09bF38AeC3A193Be08d87a32AEba385`
Human-capped launch	`0x2Bf1A67580309fC717581b171DD17A79D1683B7c`

Table 6: Main Sepolia deployments.

Parameter	**Hook pools**	**Launches**
Ladder step ε	0.1%	1.5%
Supply (settled demo)	n/a	1,000,000 plus 250,000 for the pool
Share of proceeds to the pool	n/a	40%
Batch window / sale length	5 blocks	14 days (open launches)
Rescue window	1,800 blocks	1 day
Max orders / bids	32	40
Depth cap per side	5% of reserves	n/a
Price range	spot, up to P_B	\$0.10 to \$2.00 (settled demo), \$0.05 to \$1.00 (open)
Pool fee, liquidity	0, full range	0, full range, locked

Table 7: Parameters used in the Sepolia deployment.

The repository contains the paper we implement (`docs/2610.36629.pdf`), the Python research code and its findings (`research/`), the contracts and tests (`contracts/`), and the app and keeper (`app/`).